Algorithms – Final Exam

12.3.23 – Term B

Prof. Tami Tamir

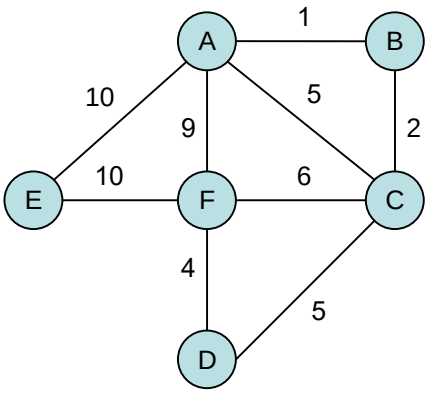
I.D:_____

- Exam length: 3 hours
- Allowed open material: Attached fact-sheet + one reference paper.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).

G O O D L U C K !

Question 1 (20 pts.)

♣ **1.1** (8 pts.) An undirected weighted graph is given in the figure. Complete (no need to justify):



a. The weight of **every minimum spanning tree** of the graph: ☐

b. The number of different MSTs: ☐

c. After Kruskal's algorithm considers the first **three** edges, the partition of the vertices into union-find sets is:

☐

d. Prims' Algorithm is performed on the graph starting from the vertex F. A possible content of the best-edge array after **two** edges are added to T is:

vertex	A	B	C	D	E	F
To						
Cost						

♣ **1.2** (12 pts.) Given an undirected simple connected graph $G = (V, E)$, with a weight function $w: E \to \{7,13\}$, that is, the weight of each edge is 7 or 13. **Suggest an algorithm** whose running time is $O(|V| + |E|)$, which returns the weight of a minimum spanning tree of G. **Prove** the correctness of the algorithm and **justify** the time complexity.

Note that there is no need to return an MST, but only the weight of an MST.

Question 2 (3*4=12 pts.)

Let G=(V,E) be a **simple directed** graph over $|V| \geq 2$ vertices.
Prove or disprove each of the following three claims (separately):

♣ **2.1** If the underlying undirected graph of G is connected, and in addition, for every node $v \in V$, it holds that in-degree(v)≥1 and out-degree(v)≥1, then G is strongly connected.

♣ **2.2** If G has a directed Euler cycle, then there is a DFS run on G that produces a spanning tree which is a simple directed path.

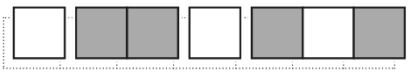
♣ **2.3** If there are two distinct nodes a,b, such that in every run of DFS(a) and in every run of DFS(b), the resulting spanning forest consists of a single tree, then G has no topological sort.

Question 3 (18 pts.)

Dana needs to tile a long path composed of a sequence of n squares. Each square in the path will be blue or white, and the tiling plan is given by an array P of length n, where for every $1 \le i \le n$, $P[i] \in \{B,W\}$ and indicates whether the i-th square should be blue (B) or white (W). Dana has tiles of three types and in limited quantities. She has:
T_1 tiles of size 1x1 colored white (W)
T_2 tiles of size 1x2 in which the two squares are blue (BB)
T_3 tiles of size 1x3 whose three squares form the sequence blue-white-blue (BWB)

Suggest an algorithm, based on **dynamic programming**, that determines whether Dana can tile the path with the tiles she has. Dana cannot cut tiles and may not lay tiles that together include more than n squares. Note: your algorithm should return True or False, no need to return the corresponding sequence of tiles.

Example: suppose n=7, P=WBBWBWB. If $T_1 = 3, T_2 = 2, T_3 = 1$, then Dana can build the path using tiles of type 1,2,1,3 (see figure).
If $T_1 = 1, T_2 = 2, T_3 = 3$, then Dana cannot build the required path.



For each $1 \le j \le n, 0 \le a_2 \le T_2, 0 \le a_3 \le T_3, 0 \le a_1 \le T_1$, let $Tile[j, a_1, a_2, a_3]$ be a boolean variable whose value is True iff Dana can tile a path whose tiling plan is given by the sub-array of P that spans from P[1] to P[j], if she has a_k tiles of type k (for $1 \le k \le 3$).

3.1 (2 pts.) What is the expression describing the solution's value (no need to explain)?

♣ **3.2** (12 pts.) Write a dynamic programming formula for calculating $Tile[j, a_1, a_2, a_3]$. Distinguish between the base and the general case. Explain the formula in words.

♣ **3.3** (4 pts.) Describe the DP-table, its size, and the order it is filled. Analyze the time-complexity of the algorithm as a function of n, T_1, T_2, T_3. Justify your answer.

Question 4 (20 pts.)

4.1 (2*4=8 points). Let G=(V,E) be a **flow network**. G
The graph contains a directed path $p = a \to b \to c \to d$ (there may by additional vertices and edges in G). The capacity of each of the three edges in p is 6. For each of the following descriptions (separately), determine whether it is possible. If yes - justify with an example (draw the network before and after the change, and specify why the description holds). If not - explain why.

♣ a. If the capacity of **each** of the edges in p increases from 6 to 8, then the maximum flow value in G increases by 4.

♣ b. If the capacity of **one** edge in p decreases from 6 to 4, then the maximum flow value in G decreases by 3.

♣ **4.2** (12 pts) Let $G = (V, E)$ be a flow network with source and sink nodes $s, t \in V$, and a capacity function where $c(e) > 1$ for every $e \in E$. It is known that s is a root of G, and the out-degree of t is 0.
For each node $v \in V$ define $c_{in}(v) = \sum_{u \in V} c(u, v)$ and $c_{out}(v) = \sum_{u \in V} c(v, u)$. That is, $c_{in}(v)$ and $c_{out}(v)$ are, respectively, the sums of the capacities of the edges entering and leaving v.
Let $x \in V \setminus \{s, t\}$. It is given that for each vertex $v \in V \setminus \{s, t, x\}$, it holds that $c_{in}(v) = c_{out}(v)$, and for the vertex x, $c_{in}(x) = c_{out}(x)$+ 1.
Suggest an algorithm, whose running time is $O(|V| + |E|)$, for finding a maximum flow in the network. Explain shortly why the flow is legal and why it is a maximum flow.



♣ Question 5 (16 pts.)

Let $G = (V, E)$ be a directed graph with a positive weight function on the edges $w: E \to \mathbb{R}^+$. Let $s, t, a \in V$ be three distinct vertices in G. The graph describes a network of roads and the weight of each edge describes the traveling time on the corresponding road. There are $deg_{out}(a) > 1$ roads that leave the vertex a. Yariv wants to get from node s to node t in a shortest path. You want to **delay** him as much as possible. You can block all the roads leaving vertex a except for one road that will remain open. **Suggest an efficient algorithm** that finds the road that leaves a and will remain open, so that Yariv's shortest path from s to t is as long as possible. The algorithm should return the corresponding edge. **Justify** shortly the correctness of your algorithm, and **analyze** the time complexity

Clarification: Yariv does not have to pass through vertex a, but if he passes through it, he must continue on the edge you choose.

Question 6 (14 pts.)

♣ **6.1** (7 pts.) The algorithm for finding a Hoffmann code is performed on a text above the alphabet $\Sigma=\{a,b,c,d,e,f\}$. It is known that $p(a) \geq p(b) \geq p(c) \geq p(d) \geq p(e) \geq p(f)$. The table shows some of the input and output data of the run. Complete the missing entries in the table. That is, you should determine frequencies for the characters b,d,e,f, and optimal code words for the characters a,c,f, so that the table contains possible input and output of the algorithm.

No need to justify

letter	**a**	**b**	**c**	**d**	**e**	**f**
frequency	0.5		0.1			
code		10		1101	1110	

♣ **6.2** (7 pts.) Clarification: This section is **not** a continuation of section 6.1.

Let a,b be two characters for which $p(b) \leq p(a) \leq 2 \cdot p(b)$ in some text (which may contain additional characters besides a,b). Denote by hof(c) the code-word given to the character c in some optimal Hoffman code for the text, and let |hof(c)| denote the length of hof(c).

Prove or disprove: $|hof(b)| \leq |hof(a)| + 3$.

Spare page – use if needed.

